

Practical Question

Practical: Build a RESTful API using Express.js

Create a **Node.js Express** application that manages a simple **Student Management API**.

Requirements:

Your application must:

1. Use **Express.js framework**
2. Use **dotenv** for environment configuration
3. Implement **middleware**:
 - o Built-in middleware (`express.json`)
 - o Custom middleware (request logger)
 - o Third-party middleware (`morgan`)
4. Create **RESTful API endpoints** using:
 - o GET
 - o POST
 - o PUT
 - o DELETE
5. Organize routes using **Express Router modules**
6. Implement **proper error handling**
7. Run the application on a port defined in `.env`

STEP 1: What are we building?

We are building a **simple REST API using Express.js** that:

- Uses **Express framework**
- Uses **middleware**
- Uses **REST methods (GET, POST, PUT, DELETE)**
- Uses **Router modules**
- Uses **dotenv for environment configuration**

- Handles **errors properly**

The API will manage **students**.

STEP 2: Create Project Folder

1. Open **VS Code**
2. Open **Terminal** (Ctrl + `)
3. Type:

```
mkdir student-api
```

```
cd student-api
```

 This creates a folder for our project.

STEP 3: Initialize Node.js Project

In terminal:

```
npm init -y
```

 This creates `package.json` (required for Node projects).

STEP 4: Install Required Packages

```
npm install express dotenv morgan
```

Why these packages?

Package	Purpose
express	Web framework
dotenv	Load environment variables
morgan	Third-party logging middleware

STEP 5 (VERY CLEAR): Creating Files & Folders in VS Code

At this step, **we are NOT writing code yet**.
We are **only creating folders and files** inside VS Code.

◆ What is Step 5 about?

We want this structure:

```
student-api
|— server.js
|— .env
|— routes
|   └─ students.js
|— middleware
|   └─ logger.js
```

This means:

- `server.js` → main Express file
 - `.env` → environment variables
 - `routes` → folder for routes
 - `middleware` → folder for middleware
-



Step 5.1: Open the Project Folder in VS Code

1. Open **VS Code**
2. Click **File** → **Open Folder**
3. Select the **student-api** folder
4. Click **Open**

👉 Now you will see **student-api** in the left side (Explorer panel)



Step 5.2: Create `server.js` File

1. In **left Explorer panel**
2. Right-click on **student-api**
3. Click **New File**
4. Type:

`server.js`

5. Press **Enter**

✓ `server.js` is created



Step 5.3: Create `.env` File

1. Right-click on **student-api**
2. Click **New File**
3. Type exactly:

`.env`

4. Press **Enter**

⚠ Important:

- The file **must start with a dot**
- It is **not** `.env.txt`

✓ `.env` file is created



Step 5.4: Create `routes` Folder

1. Right-click on **student-api**
2. Click **New Folder**
3. Type:

`routes`

4. Press **Enter**

✓ `routes` folder is created



Step 5.5: Create `students.js` Inside `routes`

1. Click on **routes** folder
2. Right-click on **routes**
3. Click **New File**
4. Type:

`students.js`

5. Press **Enter**



`students.js` is created inside `routes`



Step 5.6: Create `middleware` Folder

1. Right-click on **student-api**
2. Click **New Folder**
3. Type:

`middleware`

4. Press **Enter**



`middleware` folder is created



Step 5.7: Create `logger.js` Inside `middleware`

1. Click on **middleware**
2. Right-click on **middleware**
3. Click **New File**
4. Type:

`logger.js`

5. Press **Enter**



`logger.js` is created

✅ Step 5 DONE 🎉

Now your VS Code **must look like this**:

```
student-api
├─ server.js
├─ .env
├─ routes
│   └─ students.js
└─ middleware
    └─ logger.js
```

If this structure matches, **you have done Step 5 correctly** ✅

🟢 STEP 6: Write Inside `.env` File

👉 Click on the `.env` file in VS Code.

Type this **one line only**:

```
PORT=3000
```

What this means:

- We tell our server to run on **port 3000**
- This value will be used in `server.js`

📌 Save the file (Ctrl + S)

🟢 STEP 7: Write Code in `server.js`

👉 Click on **server.js**

Now write the code **step by step**.

◆ STEP 7.1: Import Required Packages

```
const express = require('express');
```

```
const dotenv = require('dotenv');  
const morgan = require('morgan');
```

Simple meaning:

- `express` → create server
 - `dotenv` → read `.env`
 - `morgan` → show request logs
-

◆ STEP 7.2: Import Student Routes

```
const studentRoutes = require('./routes/students');
```

Meaning:

- Connects `students.js` file to server
-

◆ STEP 7.3: Load Environment Variables

```
dotenv.config();
```

Meaning:

- Reads `PORT` from `.env`
-

◆ STEP 7.4: Create Express App

```
const app = express();
```

Meaning:

- Creates Express application
-

◆ STEP 7.5: Built-in Middleware

```
app.use(express.json());
```

Meaning:

- Allows server to accept JSON data

- Required for POST & PUT
-

◆ STEP 7.6: Third-party Middleware

```
app.use(morgan('dev'));
```

Meaning:

- Shows request details in terminal
-

◆ STEP 7.7: Connect Routes

```
app.use('/api/students', studentRoutes);
```

Meaning:

- All student routes start with /api/students
-

◆ STEP 7.8: Error Handling Middleware

```
app.use((err, req, res, next) => {  
  res.status(500).json({  
    message: err.message  
  });  
});
```

Meaning:

- Handles server errors safely
-

◆ STEP 7.9: Start Server

```
const PORT = process.env.PORT || 5000;  
  
app.listen(PORT, () => {  
  console.log(`Server running on port ${PORT}`);  
});
```


Meaning:

- Starts server on port 3000
-



server.js is DONE

Save the file (Ctrl + S)



STEP 8: Write Custom Middleware (`logger.js`)



Click on **middleware/logger.js**

Type:

```
const logger = (req, res, next) => {  
  console.log("Request Method:", req.method);  
  console.log("Request URL:", req.url);  
  next();  
};
```

```
module.exports = logger;
```

Meaning:

- Logs request info
- `next()` sends request forward

Save the file 



STEP 9: Write Routes (`students.js`)



Click on **routes/students.js**

◆ STEP 9.1: Import Express & Logger

```
const express = require('express');
```

```
const router = express.Router();

const logger = require('../middleware/logger');
```

◆ STEP 9.2: Student Data

```
let students = [

  { id: 1, name: "Amit", course: "Node.js" },

  { id: 2, name: "Riya", course: "Express.js" }

];
```

◆ STEP 9.3: GET All Students

```
router.get('/', logger, (req, res) => {

  res.json(students);

});
```

◆ STEP 9.4: GET Student by ID

```
router.get('/:id', (req, res) => {

  const student = students.find(s => s.id == req.params.id);

  if (!student) {

    return res.status(404).json({ message: "Student not found" });

  }

  res.json(student);

});
```

◆ STEP 9.5: POST Student

```
router.post('/', (req, res) => {

  const newStudent = {
```

```
      id: students.length + 1,

      name: req.body.name,

      course: req.body.course

    });

    students.push(newStudent);

    res.status(201).json(newStudent);

  });
```

◆ STEP 9.6: PUT Student

```
router.put('/:id', (req, res) => {

  const student = students.find(s => s.id == req.params.id);

  if (!student) {

    return res.status(404).json({ message: "Student not found" });

  }

  student.name = req.body.name;

  student.course = req.body.course;

  res.json(student);

});
```

◆ STEP 9.7: DELETE Student

```
router.delete('/:id', (req, res) => {

  students = students.filter(s => s.id != req.params.id);

  res.json({ message: "Student deleted successfully" });

});
```

```
module.exports = router;
```

Save the file 

STEP 10: Run the Server

Open **Terminal** in VS Code and type:

```
node server.js
```

You should see:

```
Server running on port 3000
```

STEP 11: Test in Browser

Open browser and go to:

```
http://localhost:3000/api/students
```

You will see student data.

SIMPLE FINAL UNDERSTANDING

File	Purpose
.env	Port number
server.js	Main server
logger.js	Custom middleware
students.js	API routes

SHORT EXPLANATION OF CODE:

Step-by-Step Solution (VS Code)

Step 1: Create Project Folder

```
mkdir student-api
cd student-api
```

Open folder in VS Code.

Step 2: Initialize Node Project

```
npm init -y
```

Step 3: Install Required Packages

```
npm install express dotenv morgan
```

Step 4: Project Structure

```
student-api
├── server.js
├── .env
├── routes/
│   └── students.js
├── middleware/
│   └── logger.js
```

Step 5: Environment Configuration (.env)

Create a .env file:

```
PORT=3000
```

Step 6: Create Main Server File (server.js)

```
const express = require('express');
const dotenv = require('dotenv');
const morgan = require('morgan');
const studentRoutes = require('./routes/students');

dotenv.config();
```

```

const app = express();

// Built-in middleware
app.use(express.json());

// Third-party middleware
app.use(morgan('dev'));

// Routes
app.use('/api/students', studentRoutes);

// Error handling middleware
app.use((err, req, res, next) => {
  res.status(500).json({
    success: false,
    message: err.message
  });
});

// Start server
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});

```

Step 7: Custom Middleware (`middleware/logger.js`)

```

const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
};

module.exports = logger;

```

Step 8: Create Router Module (`routes/students.js`)

```

const express = require('express');
const router = express.Router();
const logger = require('../middleware/logger');

// Sample data
let students = [
  { id: 1, name: 'Rahul', course: 'Node.js' },
  { id: 2, name: 'Anita', course: 'Express.js' }
];

// GET all students
router.get('/', logger, (req, res) => {
  res.json(students);
});

// GET student by ID
router.get('/:id', (req, res) => {
  const student = students.find(s => s.id === req.params.id);
  if (!student) {
    return res.status(404).json({ message: 'Student not found' });
  }
  res.json(student);
});

```

```

});

// POST new student
router.post('/', (req, res) => {
  const newStudent = {
    id: students.length + 1,
    name: req.body.name,
    course: req.body.course
  };
  students.push(newStudent);
  res.status(201).json(newStudent);
});

// PUT update student
router.put('/:id', (req, res) => {
  const student = students.find(s => s.id == req.params.id);
  if (!student) {
    return res.status(404).json({ message: 'Student not found' });
  }

  student.name = req.body.name;
  student.course = req.body.course;
  res.json(student);
});

// DELETE student
router.delete('/:id', (req, res) => {
  students = students.filter(s => s.id != req.params.id);
  res.json({ message: 'Student deleted successfully' });
});

module.exports = router;

```

Step 9: Run the Application

node server.js

OR (recommended):

npx nodemon server.js

Step 10: Test API Endpoints (Postman / Browser)

Method	URL	Description
GET	/api/students	Get all students
GET	/api/students/1	Get student by ID
POST	/api/students	Add student
PUT	/api/students/1	Update student
DELETE	/api/students/1	Delete student

Outcome

- ✓ Express Application Created
- ✓ Middleware Used (Built-in, Custom, Third-party)
- ✓ RESTful Routing Implemented
- ✓ Router Modules Used
- ✓ Error Handling Added
- ✓ Environment Variables Configured